



ELEC3609/9609 - Week 7 Lab

06 DRF and Django Advance

In this lab, you will explore Django REST Framework (DRF) and several advanced topics relevant to building scalable and modern web applications. The focus extends beyond basic API usage to include stateless authentication with JWT, asynchronous task handling with Celery, real-time communication with WebSockets, and performance considerations like the N+1 query problem. These features form the backbone of robust backend systems in real-world production environments.

Individual Task: Individual Task 6 - Find and Fix a Hidden N+1 Problem

Lab Resource: TIL-Lab06 (Today I Learned project) source code.

Learning Objectives

After completing this lab, students should be able to:

- Understand the structure and benefits of Django REST Framework (DRF) for building APIs.
- Describe the principles of stateless authentication using JSON Web Tokens (JWT).
- Identify when and why to use asynchronous task queues such as Celery.
- Recognise the use of WebSocket for real-time communication and how Django Channels enables this.
- Understand the N+1 query problem and apply optimisation strategies using Django QuerySets.

Chapter 1. Django REST Framework (DRF)

Nowadays, developers need to support web clients, native mobile apps, and business-to-business applications. Django REST Framework (DRF) is a powerful toolkit for building Web APIs in Django. It provides a flexible and powerful system for building and consuming RESTful APIs with powerful authentication and authorization.

**TIP**

In TIL project, we create a new application `til_api` to hold all contexts for RESTful APIs. Check `api_endpoints.json` to play with TIL APIs.

Read more about [Django REST framework's API Guide](#).

1.1 Serialization

Serialization in DRF allows complex data types, such as Django (ORM) model instances and querysets, to be converted into native Python data types that can then be easily rendered into JSON or other content types. Serializers also provide deserialization, allowing parsed data to be converted back into complex types.

The serializers in REST framework work very similarly to Django's `Form` and `ModelForm` classes. This is the real "magic" that Django REST Framework provides for us.

Basic Serializer Example

```
from rest_framework import serializers
from til_app.models import Post

class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = ['id', 'subject', 'contents', 'created_at', 'visibility',
                  'tags', 'image']
```

- `PostSerializer`: This defines a serializer for the `Post` model. By inheriting from `ModelSerializer`, it automatically generates fields based on the model.
- REST framework includes both `Serializer` and `ModelSerializer` classes. `ModelSerializer` classes are simply a shortcut for creating serializer classes with default implementations of `.create()` and `.update()`.
- `class Meta`: This inner class provides metadata about this serializer.
 - `model = Post`: Specifies the model to serialize.
 - `fields = [...]`: Lists the fields to include in the serialized representation.

Serializing Objects

```
post = Post.objects.first()
serializer = PostSerializer(post) # Use PostSerializer to serialize a post.
print(serializer.data)
```

- `serializer = PostSerializer(post)`: Creates a serializer instance for a single `Post` object.
- `serializer.data`: Converts the `Post` object into a dictionary format suitable for rendering into JSON or other formats. The output will look like:

```
{
  "id": 1,
  "subject": "First Post",
  "contents": "This is the content of the first post.",
  "created_at": "2024-09-25T16:24:38.096845+10:00",
  "visibility": true,
  "tags": [
    {
      "id": 1,
      "name": "tag1",
      "creation_date": "2024-09-25T16:18:22.201810+10:00"
    }
  ],
  "image": "image_media_url"
}
```

Deserializing Objects

```
serializer = PostSerializer(data=request.data)
if serializer.is_valid():
    validated_data = serializer.validated_data
    print(validated_data)
```

- `serializer = PostSerializer(data=data)`: Creates a serializer instance with raw input data.
- `serializer.is_valid()`: Validates the input data against the serializer's fields and validation rules.
- `serializer.validated_data`: Contains the validated data in forms of dictionary, which

can be used to create or update a model instance.

Saving Instances

```
class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = ['subject', 'contents', 'visibility']

    def create(self, validated_data):
        # Custom creation logic, additional processing before saving
        return Post.objects.create(**validated_data)

serializer = PostSerializer(data=request.data)
if serializer.is_valid():
    post = serializer.save() # Calls the create() method
    print(post)
```

- `def create(self, validated_data)`: This method is called when `serializer.save()` is used with no instance.
- The `validated_data` is passed as a dictionary containing the validated input data.
- `serializer.save()`: Calls the `create()` methods and returns the newly created `Post` object.

Validation

```
class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = ['subject', 'contents', 'visibility']

    def validate_subject(self, value):
        if len(value) < 5:
            raise serializers.ValidationError("The subject must be at least 5 characters long.")
        return value
```

- You can specify custom field-level validation by adding `.validate_<field_name>` methods to `Serializer` subclass.
- These methods are similar to `.clean_<field_name>` methods on Django forms.

- These methods take a single argument, which is the field value that requires validation.

```
class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = ['subject', 'contents', 'visibility']

    def validate(self, data):
        if 'spam' in data['contents'].lower():
            raise serializers.ValidationError("The content contains
prohibited words.")
        return data
```

- You can validate the entire object by overriding the `validate()` method as object-level validation.
- This method takes a single argument, which is a dictionary of field values.
- It should raise a `serializers.ValidationError` if necessary, or just return the validated values.

1.2 Request and Response

DRF provides enhanced Request and Response objects, making it easier to handle API requests and responses.

Request Example

```
class ExampleView(APIView):
    def post(self, request):
        data = request.data
        return Response({'received_data': data})
```

- `request.data`: Contains the parsed data from the body of the request. For POST, PUT, and PATCH requests, `request.data` is a dictionary-like object containing the request data (`<QueryDict>`).
- `request.data` includes all parsed content, including *file* and *non-file* inputs.

Response Example

```
def example_function_view(request):  
    data = {'message': 'Hello, world!'}  
    return Response(data)
```

1.3 Class-Based Views and Generic API Views

REST framework provides an `APIView` class, which subclasses Django's `View` class.

```
class HelloWorldView(APIView):  
    def get(self, request):  
        return Response({"message": "Hello, world!"})
```

- `APIView`: The base class for all DRF views.
- `def get(self, request)`: Defines the behaviour for HTTP GET requests.

REST framework provides a number of pre-built views that provide for commonly used patterns. The generic views provided by REST framework allow you to quickly build API views that map closely to your database models.

```
class PostListView(generics.ListAPIView):  
    queryset = Post.objects.all()  
    serializer_class = PostSerializer
```

- `ListAPIView`: A generic view for listing multiple objects.
- `queryset`: The set of objects to be listed.
- `serializer_class`: The serializer class used to format the response data.

1.4 Custom Function-Based Views

REST framework also allows you to work with regular function based views. Function-based views offer more control and are suitable for simple or highly customised logic.

```
@api_view(['GET', 'POST'])  
@permission_classes([IsAuthenticated])  
def post_list_create(request):
```

```
if request.method == 'GET':
    posts = Post.objects.all()
    serializer = PostSerializer(posts, many=True)
    return Response(serializer.data)

elif request.method == 'POST':
    serializer = PostSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)
```

- `@api_view(['GET', 'POST'])`: Specifies that this view only handles GET and POST requests.
- `@permission_classes([IsAuthenticated])`: Restricts access to authenticated users only.

1.5 Pagination

REST framework includes support for customisable pagination styles, making large datasets easier to manage.

Global Pagination Settings

```
# settings.py
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
    'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
}
```

- The pagination style may be set globally.
- Pagination is only performed automatically if you are using the generic views or viewsets.

Custom Pagination Classes

```
from rest_framework.pagination import PageNumberPagination

class CustomPagination(PageNumberPagination):
```

```
page_size = 5
page_size_query_param = 'page_size'
max_page_size = 100
```

- You can override the pagination classes to modify particular aspects of the pagination style.

Applying Pagination to Views

```
class PostListView(generics.ListAPIView):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    pagination_class = CustomPagination
```

- `pagination_class = CustomPagination`: Specifies the custom pagination class for this view.

1.6 Authentication, Permissions and Throttling

Authentication in DRF is used to verify the identity of the user making a request. This ensures that only authorized users can access or modify resources. DRF provides several authentication mechanisms, including session-based, token-based, and JWT-based authentication.

Read more about [Django REST Framework's Authentication](#).

Permissions determine whether a request should be granted or denied. DRF provides a variety of permission classes, which can be applied globally or at the view level.

Read more about [Permissions in Django REST Framework](#).

Throttling is a mechanism used to control the rate of requests that a user or client can make to an API. It helps prevent abuse, such as spamming or excessive usage, by limiting the number of API requests within a specified time period.

Read more about [Throttling in Django REST Framework](#).

Chapter 2. JSON Web Token (JWT)

JWT (JSON Web Token) is a compact, URL-safe means of representing claims between two parties. It's a popular format used for securely transmitting information as a JSON object, and it is signed using a secret or public/private key pair to ensure the integrity and authenticity of the claims.

JWTs are powerful tools for authentication and authorization, especially in stateless and distributed environments. They offer scalability and flexibility but require careful handling to address security concerns. They are not suitable for scenarios requiring immediate revocation or one-time use, where traditional tokens stored in a database are more appropriate. Understanding when and how to use JWTs effectively is crucial for building secure and robust applications.

Read more about [Introduction to JSON Web Tokens](#) and [RFC 7519](#).

Structure of a JWT: A JWT consists of three parts separated by dots (.):

```
header.payload.signature / xxxxx.yyyyy.zzzzz
```

- **Header:** Contains metadata about the token, such as the signing algorithm and token type.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **Payload:** Contains the claims, which are statements about the user or additional data. Common claims include `iss` (issuer), `exp` (expiration time), `sub` (subject), and `aud` (audience).

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

- **Signature:** Ensures the token hasn't been altered. It's created by encoding the header and payload, then signing it with a secret key or a private key.

```
HMACSHA256(
  base64UrlEncode(header) + "." +
```

```
base64UrlEncode(payload),  
secret)
```

JWT Workflow:

- The server generates a token upon user authentication and returns it to the client.
- The client includes this token in the `Authorization` header of every subsequent HTTP request, using `Bearer` schema.
- The server validates the token by checking the signature and ensures it hasn't expired. If valid, the request is processed; otherwise, it's rejected.

Example of JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9  
.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ  
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

2.1 JWT vs. Session

Feature	JWT	Session
Storage	Client-side (usually in local storage or cookies)	Server-side (stored in memory or database)
State Management	Stateless (doesn't require server storage)	Stateful (requires server storage and session management)
Scalability	Highly scalable; no need to synchronize state across servers	Can be challenging to scale due to centralized state
Performance	Less server load; token verification is local	Higher server load; session lookup on each request
Expiration	Expiry is built into the token (<code>exp</code> claim)	Controlled by server; session can be invalidated at any time

Feature	JWT	Session
Revocation	Difficult without additional mechanisms (e.g., blacklists)	Easy; invalidate session on server
Security Concerns	Vulnerable to token theft and misuse if not properly stored; difficult to revoke	Easier to revoke; vulnerable to session hijacking
Use Case	Ideal for APIs, microservices, and single-page applications	Best for traditional web apps with server-side rendering

WARNING

Naive JWT implementation without additional measures is not a good practice for authorization. One good practice is to use token blacklist and short-lived `accessToken` with a `refreshToken`.

INFO

JWT alone is not sufficient for comprehensive authorization requirements in modern applications, and that's one of the reasons why OAuth 2.0 is often needed. Read more about [OAuth 2.0](#) and [RFC 6749](#).

2.2 JWT Is Not a Traditional Token

Why JWT Is Not a Traditional Token

- Self-Contained and Stateless:** Unlike traditional tokens, JWTs do not require server-side storage, making them stateless and self-contained. All necessary information is embedded within the token itself.
- Not Meant for One-Time Expiration:** Traditional tokens like email verification or password reset tokens are one-time-use and managed on the server. JWTs, however, are often used for continuous sessions with a defined expiration and are not inherently one-time use.
- Non-Revocable by Default:** Once issued, a JWT remains valid until it expires. Traditional

tokens stored in the database can be easily invalidated or revoked by changing their state in the database.

Use Cases for Traditional Tokens

- **Email Verification:** A one-time use token stored in the database, expiring after a single use or a set period.
- **Password Reset:** A temporary token allowing a user to reset their password. Typically expires after a few minutes.
- **Coupon or Discount Codes:** Tokens representing special offers, redeemed only once.

Summary: JWT vs. Traditional Tokens

Feature	JWT	Traditional Tokens
Storage	Client-side, no server storage needed	Server-side, stored in a database
Revocation	Difficult to revoke; requires blacklist mechanism	Easy to revoke; change state in database
Expiration	Controlled by <code>exp</code> claim; not one-time use	Usually one-time use or short-lived
Security Considerations	Token theft risk; secure storage required	Secure access to database; prevent replay attacks
Use Case	Authentication and continuous session management	One-time actions like email verification, password reset

2.3 Simple JWT

Simple JWT provides a JSON Web Token authentication backend for the Django REST Framework. It aims to cover the most common use cases of JWTs by offering a conservative set of default features. It also aims to be easily extensible in case a desired feature is not present.

`t1l_api` in this TIL demo is integrated with Simple JWT as authentication backend. Read

more about [Simple JWT](#).

Chapter 3. N+1 Problem

There is an ORM performance problem so common that it has a fancy name: the "N+1 query problem". This problem occurs when you build a query with the ORM that inadvertently causes additional queries to look up related records that were not loaded by the original query. It typically occurs when a separate database query is executed for each object in a set, resulting in N queries, where N is the number of objects, plus on initial query to retrieve the set itself. This results in a total of $N+1$ queries (a loop).

Suppose you have a list of posts, and for each post, you want to retrieve its associated comments. If you write a separate query to fetch comments (or creator's name, comment count) for each post, you end up with the following:

1. One query to fetch all posts.
2. N additional queries to fetch comments for each post.

```
# This results in N+1 trips to the database
posts = Post.objects.all() # 1 hit
for post in posts: # N hits
    comments = Comment.objects.filter(post=post)
    # creator_name = post.creator.username # second N
    # comment_count = Comment.objects.filter(post=post).count() # third N
```

The main issue with the N+1 problem is the performance bottleneck it creates due to multiple queries. As the number of objects (N) increases, the number of queries increases linearly, leading to significant slowdowns:

- **Higher Resource Consumption:** The database and the application have to communicate multiple times, degrading the overall performance (e.g., CPU usage, memory consumption, and I/O).
- **Increased Latency:** Every query incurs a delay due to the time it takes for the request to travel to the server and back (Round-Trip Time, RTT).

! INFO

Modern databases with sophisticated query optimizers can handle complex queries

efficiently. Read more about [The SQLite Query Optimizer Overview](#).

WARNING

Remember, premature optimisation is bad. If your site is small- or medium-sized and the pages are loading fine, then it's okay to ignore the optimisation.

Read more about [Django QuerySet API](#) and [Database Access Optimization](#).

3.1 Find N+1 Queries

To identify N+1 queries, you can use Django Debug Toolbar. It allows you to inspect the SQL queries generated for each request.

For more information about installation, visit the [Django Debug Toolbar Documentation](#).

INFO

This TIL project provides with two endpoints to check the N+1 problem, visit:

- **(Problematic)** http://127.0.0.1:8000/til_app/posts_n_plus_one/
- **(Optimised)** http://127.0.0.1:8000/til_app/posts_optimized/

These endpoints are implemented in `til_app.views.post_comments_n_plus_one()` and `til_app.views.post_comments_optimized()` respectively.

3.2 Fix N+1 with Optimised QuerySet

Optimised QuerySets can prevent the N+1 problem by using methods like `select_related`, `prefetch_related`, and `annotate`.

Read more about [QuerySet API reference](#).

select_related()

Use when fetching related objects via **foreign key** relationships. It performs an SQL `JOIN` operation to fetch related objects in a single query.

```
posts = Post.objects.select_related('creator').all()
for post in posts:
    print(post)
    print(post.creator.username)
```

This will return a `QuerySet` for each post with its creator.

```
SELECT "post"."id", "post"."subject", "post"."contents",
"auth_user"."username"
FROM "post"
INNER JOIN "auth_user" ON ("post"."creator_id" = "auth_user"."id")
```

prefetch_related()

Use when fetching related objects via many-to-many or reverse foreign key relationships.

```
posts = Post.objects.prefetch_related('post_comment').all()
for post in posts:
    print(post)
    for comment in post.post_comment.all():
        print(comment)
```

Django performs two queries and then "joins" the results in Python.

```
SELECT "post"."id", "post"."subject"
FROM "post";

SELECT "comment"."id", "comment"."post_id", "comment"."content"
FROM "comment"
WHERE "comment"."post_id" IN (1, 2, 3, ...); # All post IDs from the first
query
```

annotate()

`annotate()` can be used to aggregate data and perform calculations across related models without requiring additional database queries for each related object.

```
posts =
Post.objects.annotate(comment_count=models.Count('post_comment')).all()
```

```
for post in posts:
    print(post.comment_count)
```

For an in-depth discussion of aggregation, see [Aggregation in Django](#).

```
SELECT "post"."id", COUNT("post_comment"."id") AS "comment_count"
FROM "post"
LEFT JOIN "post_comment" ON ("post"."id" = "post_comment"."post_id")
GROUP BY "post"."id"
```

Perform Raw SQL Queries

Django gives the ability to write custom SQL queries for complex user cases. Read more about [Performing raw SQL queries](#).

WARNING

You should be very careful whenever you write raw SQL, for both security concerns and performance.

3.3 Still N+1?

Sometimes, you cannot avoid N+1 problems entirely. Here are some strategies to mitigate performance issues.

Pagination (Infinite scroll)

Load a few items at a time using Django's built-in pagination utilities.

```
from django.core.paginator import Paginator

paginator = Paginator(Post.objects.all(), 9) # Show 9 posts per page
page_number = request.GET.get('page')
page_obj = paginator.get_page(page_number)
```

Check `til_app.views.posts()` and `template/post/posts.html` to see how to implement a simple pagination with infinite scroll. Read more about [Django Pagination](#).

Lazy Loading

Load only the data needed at a given time, avoiding unnecessary data fetching. This is commonly used in frontend optimisation for images, videos, etc. Learn more about [Lazy Loading](#).

Caching in Django

Use caching mechanisms to store frequently accessed data and reduce database hits. Read more about [Django's cache framework](#).

❗ OPTIONAL READING – NOT ASSESSED IN FINAL EXAM

The following readings (Chpater 4 + 5) are **optional**. They are provided for your interest and broader topics, but **they will not be assessed in the final exam**.

Chapter 4. Asynchronous Task Queues

An asynchronous task queue is one where tasks are executed at a different time from when they are created, and possibly not in the same order they were created. This mechanism allows you to execute background tasks outside the main application process, enabling non-blocking operations.

4.1 Basic Concepts and Definitions

1. **Task/Job:** A unit of work to be performed asynchronously. It could be anything from sending an email to processing a file.
2. **Task Queue:** A data structure that holds tasks to be executed.
3. **Broker/Message Queue:** A middleware component that handles the storage and delivery of tasks. Examples include Redis, RabbitMQ, and Amazon SQS.
4. **Producer:** The entity (e.g., code, web application) that creates tasks and puts them in the task queue.
5. **Worker:** A process that fetches tasks from the task queue and executes them. Multiple workers can be used to process tasks concurrently. Most commonly each worker runs as a daemon under supervision.

! INFO

"**Task queue**" is not a traditional term in academic context. In Django world, when someone writes **task queue**, they usually mean asynchronous task queue.

! INFO

The concept of a task queue is closely related to message queues and message-oriented middleware, which have their roots in distributed systems and concurrent computing.

Read more about [Message-oriented middleware](#), [Message queue](#), [Message broker](#), and [Scheduling](#).

4.2 When Do You Need a Task Queue?

You should consider using a task queue in the following scenarios:

- **Long-Running Tasks:** Tasks like sending emails, generating reports, processing large data files, or image processing should not block the main application thread.
- **Resource-Intensive Operations:** Operations that consume significant CPU or I/O resources, such as video encoding or machine learning model training.
- **Scheduled Tasks:** Automating repetitive tasks like backups, data synchronization, or periodic notifications.
- **Concurrency:** Managing concurrency to ensure that specific tasks run at controlled rates.

Possible Use Cases

Issue	Use Task Queue?	Reason
Sending bulk email	Yes	Each send involves network I/O; bulk sends are slow if done inline.
Modifying files (including images)	Yes	File and image processing is CPU/I/O intensive and may take seconds.
Fetching large amounts of data from third-party APIs	Yes	Network calls are slow and can easily exceed request timeouts.

Issue	Use Task Queue?	Reason
Inserting or updating a lot of records into a table	Yes	Bulk DB operations can be long-running; safer to run asynchronously.
Performing time-intensive calculations	Yes	CPU-bound; should not block the main web thread.
Updating a user profile	No	Simple DB write, usually completes within milliseconds.
Adding a blog	No	One insert and maybe a few updates; fast enough for request/response.

**DANGER**

Task queue is not suitable for User-Interactive tasks.

**WARNING**

Task queue can add complexity but can improve user experience. Arguably it comes down to whether a particular piece of code causes a bottleneck and can be delayed for later when more free CPU cycles are available.

4.3 Implementations in Django

Celery

A robust, distributed task queue system that supports complex workflows and high concurrency. It is ideal for handling heavy background tasks like sending bulk emails, processing large data sets, and executing scheduled jobs. Celery works with multiple brokers (e.g., Redis, RabbitMQ) and offers features like task retry, chaining, and monitoring.

This TIL project utilises Celery for asynchronous email services. Refer to the following guides for essential guidance on integrating Celery with Django:

- [Introduction to Celery](#)

- [Celery 5.4 >> Django](#)
- [Celery Workers Guide](#)
- [Celery + Redis + Django](#)



WHERE IS THE CODE?

The Celery-backed email flow in this TIL project is implemented in:

- `til_api.views.RegisterView()` — enqueues the email task during user registration.
- `til_api.tasks` — contains the Celery task(s) that actually send the email.

Explore those modules to see how the task is scheduled and executed.



SETTINGS REMINDER

This TIL project is configured for **demo purposes**:

- The `.env` file uses placeholder values for email. To make it actually work, you'll need to replace them with a real email service configuration.
- In settings, `celery_task_always_eager = True` is enabled. This means tasks run **synchronously in-process**, not in a real background worker.

If you want true asynchronous behaviour, you'll need to change these settings and run Celery with a broker (see the links above).

Django Native Asynchronous Support

Django has support for writing asynchronous ("async") views, along with an entirely async-enabled request stack if you are running under ASGI. Async views will still work under WSGI, but with performance penalties, and without the ability to have efficient long-running requests.

Read more about [Asynchronous Support](#).

Other lightweight Task Queues

- [Django Q](#)
- [Huey](#)

Chapter 5. WebSocket

WebSocket is a protocol that enables real-time, two-way communication between a client (like a web browser) and a server over a single, persistent connection. Unlike traditional HTTP, which requires separate requests for each interaction, WebSocket allows continuous, low-latency communication. This makes it ideal for use cases like chat applications, live notifications, online gaming, and collaborative tools.

The latest specification of WebSocket protocol is defined as [RFC 6455](#), which is supported by various browsers like Google Chrome, Safari, Mozilla Firefox, etc.

5.1 Django Channels

Django Channels extends the capabilities of Django beyond HTTP, enabling it to handle protocols that require long-running connections like WebSocket, MQTT, and more. It allows Django applications to manage real-time functionalities such as chat rooms, live updates, and notifications while maintaining compatibility with the standard Django ecosystem.

Key Concepts

1. **ASGI (Asynchronous Server Gateway Interface)**: Django Channels uses ASGI instead of WSGI to support asynchronous protocols. ASGI is the asynchronous counterpart of WSGI, enabling Django to handle WebSockets and other long-lived connections.
2. **Consumers**: A consumer is the basic unit of Channels code. Consumers are Django's equivalent of views for handling WebSocket and other asynchronous events. Consumers structure your code as a series of functions to be called whenever an event happens, rather than making you write an event loop. Consumers also allows you to write synchronous or async code, and deal with handoffs and threading for you.
3. **Routing**: Similar to Django's URL routing for HTTP requests, Channels uses routing to map WebSocket connections and other protocol events to specific consumers. This allows you to combine and stack your consumers (and any other valid ASGI application) to dispatch based on what the connection is.
4. **Channel Layers**: Channel layers provide a mechanism for communication between different instances of the application. It is a cross-process communication layer that supports operations like group messaging and broadcast messaging. Channel layers use backends like Redis or in-memory storage to manage communication between

consumers.



TIP

Read more about [Django Channels](#).

5.2 Building a Simple Chat Room

In this demo, we extend the TIL project to support real-time chat using **Django Channels**. The implementation combines WebSockets, a consumer, and a simple frontend template in `til_app`.

The chat can be tested at: http://127.0.0.1:8000/til_app/chat/All/

1. Settings and ASGI configuration

- `til_project/settings.py`

Add Channels and configure ASGI + channel layers:

```
INSTALLED_APPS = [  
    'daphne',  
    'channels',  
    ...  
]  
  
ASGI_APPLICATION = 'til_project.asgi.application'  
  
CHANNEL_LAYERS = {  
    "default": {"BACKEND": "channels.layers.InMemoryChannelLayer"}  
}
```

- `til_project/asgi.py`

Route HTTP to Django and WebSockets to Channels:

2. Routing and Consumers

- `til_app/routing.py`

Define WebSocket URL patterns:

- ```

websocket_urlpatterns = [
 re_path(r'ws/chat/(?P<room_name>\w+)/$',
consumers.ChatConsumer.as_asyncgi()).

class ChatConsumer(WebsocketConsumer):
 def connect(self):
 self.room_name = self.scope['url_route']['kwargs']['room_name']
 self.room_group_name = f"chat_{self.room_name}"
 async_to_sync(self.channel_layer.group_add)
 (self.room_group_name, self.channel_name)
 self.accept()

 def receive(self, text_data):
 data = json.loads(text_data)
 async_to_sync(self.channel_layer.group_send)(
 self.room_group_name,
 {"type": "chat_message", "username": data["username"],
"message": data["message"]})
)

 def chat_message(self, event):
 self.send(text_data=json.dumps(event))

```

### 3. Views and URLs

- til\_app/views.py

```

@login_required
def chat(request, room):
 messages = ChatMessage.objects.filter(room_name=room).order_by('-
timestamp')[:50]
 return render(request, "chat/index.html", {"room_name": room,
"messages": reversed(messages)})

```

- til\_app/urls.py

```

path('chat/<str:room>/', views.chat, name='chat')

```

### 4. Template and Frontend Logic

- template/chat/index.html

Connect to WebSocket and display messages dynamically:

```
const chatSocket = new WebSocket(
 (window.location.protocol === 'https:' ? 'wss://' : 'ws://')
 + window.location.host
 + '/ws/chat/' + "{{ room_name }}" + '/'
);

chatSocket.onmessage = function(e) {
 const data = JSON.parse(e.data);
 // Append new messages to chat window
};

document.querySelector('#chat-message-submit').onclick = function() {
 chatSocket.send(JSON.stringify({
 'username': '{{ user.username }}',
 'email': '{{ user.email }}',
 'message': messageInput.value
 }));
};
```

### ! SUMMARY

- `settings.py` + `asgi.py` → enable Channels and WebSocket support.
- `routing.py` + `consumers.py` → manage connections and message broadcasting.
- `views.py` + `urls.py` → serve chat pages and history.
- `index.html` → frontend UI and JS logic for sending/receiving messages.

This structure demonstrates a minimal yet functional real-time chat system with Django Channels.

## Further Readings

- Django REST Framework
- Celery
- Django Channels
- JWT, RFC 7519
- OAuth 2.0, RFC 6749

- [WebSocket, RFC 6455](#)
- [WSGI, PEP 3333](#)
- [ASGI](#)

## Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.